



MEETMIND-AI

Project Information



AUGUST 12, 2026

MUHAMMAD FAHAD

1. Project Name:

*MeetMind AI — Intelligent Meeting Transcription,
Summarization & Knowledge Retrieval System*

2. Objective

MeetMind AI is an enterprise-grade AI-powered meeting intelligence ecosystem consisting of a React Web Application, a Manifest V3 Chrome Extension, and a Python FastAPI Backend.

The platform captures, transcribes, processes, and stores meeting audio and notes in real time. Leveraging Retrieval-Augmented Generation (RAG) and structured LLM pipelines, MeetMind AI automatically generates structured meeting summaries, extracts actionable tasks, identifies key decisions, and enables contextual multi-session Q&A across meeting history.

3. Key Features

- **Multi-Channel Meeting Capture:** Capture audio and meeting context directly from browser-based platforms (Google Meet, Zoom Web, Microsoft Teams) via Chrome Extension or upload recorded audio/transcripts via the Web App.
- **Automated Audio Pipeline & Transcription:** Multi-provider Whisper and LLM integration for fast, accurate speech-to-text conversion.
- **Structured Information Extraction:** Uses strict Pydantic schemas on the backend to parse:
- **Executive Summaries:** Concise meeting overviews with main discussion points.
- **Action Items & Tasks:** Assigned owners, deadlines, and priority levels.
- **Key Decisions & Highlights:** Formal logging of decisions made during discussions.
- **RAG-Powered Meeting Chat (Vector Database):** Embedded semantic search (ChromaDB / FAISS) allowing users to query single or across historical meetings (e.g., *"What did we decide about the API budget last week?"*).
- **Multi-Provider AI Key Manager:** Dynamic fallback & switching across OpenAI, Groq, Google Gemini, and local Ollama models.



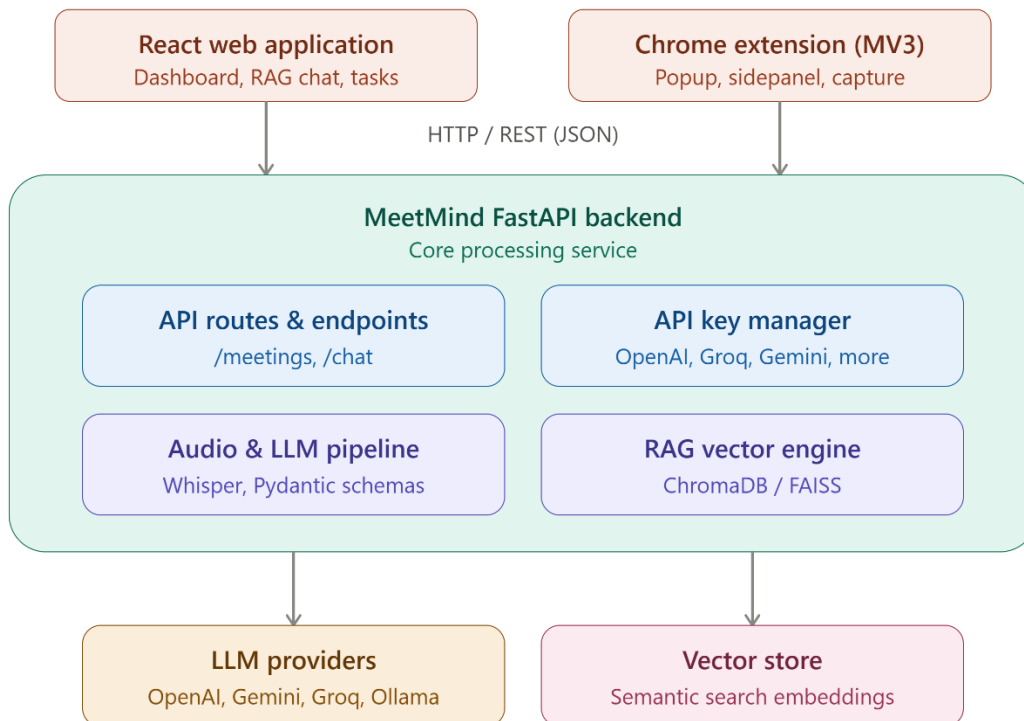
- Modern Web Interface: Dark mode glassmorphism UI built with React, TypeScript, Tailwind CSS, and Framer Motion micro-animations.
- Chrome Extension (Manifest V3): Sidepanel and popup integration providing one-click meeting controls, audio capture, and quick status monitoring.

4. Tools & Technologies

- Backend (Python API)
- Framework: Python 3.10+, FastAPI, Uvicorn
- Data Validation: Pydantic V2 (structured output parsing for LLM calls)
- AI & LLM Services: OpenAI API, Groq SDK, Google Gemini API, Ollama (Local LLM support)
- Embeddings & Vector Database: ChromaDB / FAISS, Sentence-Transformers / OpenAI Embeddings
- Audio Processing: OpenAI Whisper API / Faster-Whisper, PyDub
- Testing & Quality: PyTest, HTTPX, Logging system with detailed audit trails
- Frontend (React Web Application)
- Framework: React 18, Vite, TypeScript
- State & Forms: React Hook Form (RHF) with Zod validation schema
- Styling & UI: Tailwind CSS, Framer Motion (page transitions & micro-interactions), Lucide Icons
- API Layer: Axios / Fetch with centralized API service architecture
- Testing: Vitest, React Testing Library
- Chrome Extension
- Standard: Manifest V3 compliant
- Components: Background Service Worker, Content Scripts (DOM listener/injector), Side Panel UI & Popup
- Build System: Vite + Custom build/copy workflows with TypeScript



5. System Architecture



The diagram shows MeetMind AI's layered architecture. Users interact through two clients — a React web dashboard and a Chrome extension — both sending requests over HTTP/REST to a FastAPI backend. The backend core handles routing and API key management for multiple LLM providers (OpenAI, Groq, Gemini, Ollama). A dedicated audio and LLM pipeline processes speech using Whisper and structured Pydantic schemas, while a RAG vector engine handles semantic retrieval via ChromaDB/FAISS. These two processing modules connect outward to external LLM providers and a vector store, closing the loop between raw meeting input and searchable, structured knowledge.

6. Team Members

Team Lead / Full-Stack Engineer: *Muhammad Fahad*

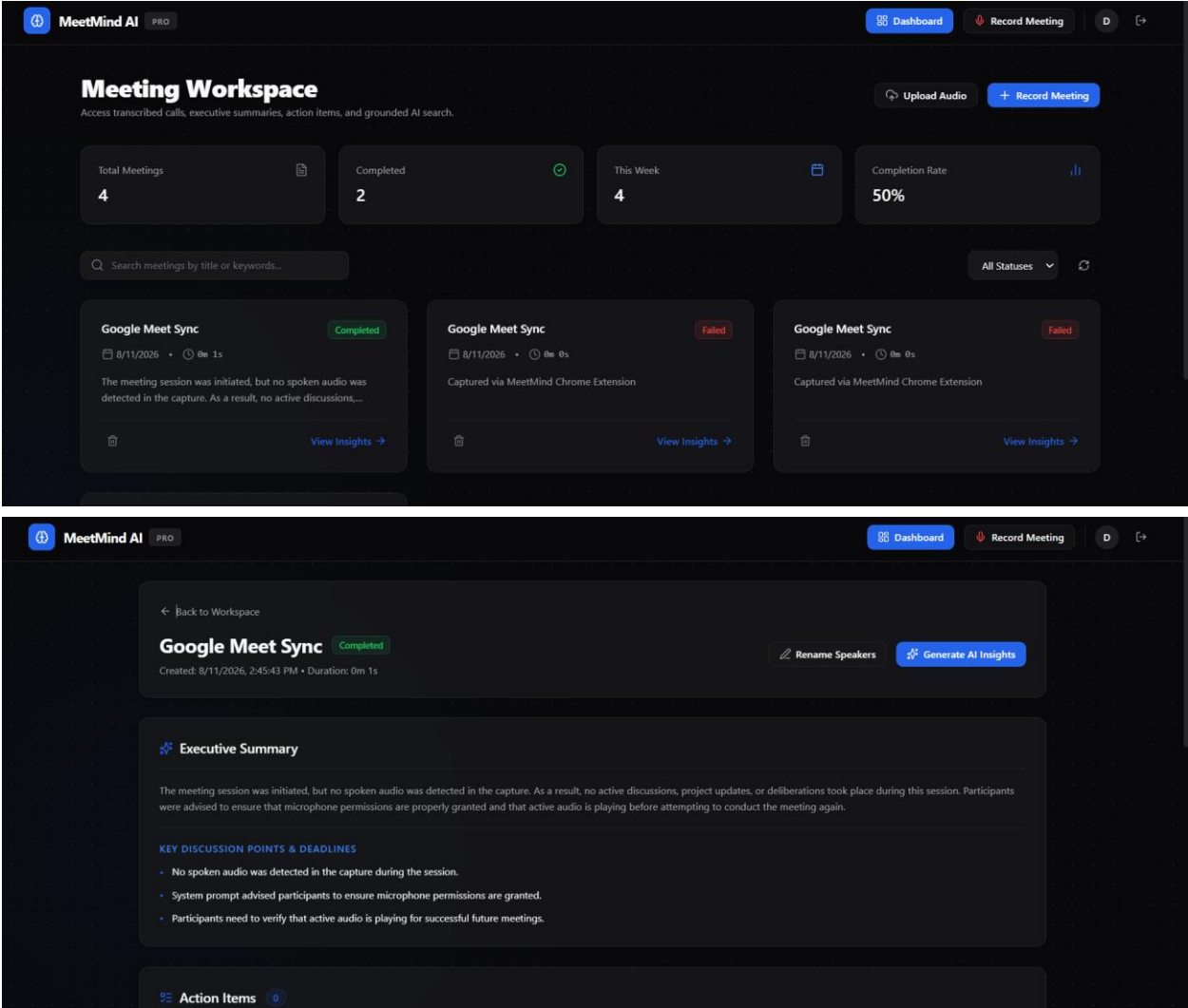


7. Current Progress & Project Status

Module	Status	Description
Backend API Core	Completed	FastAPI setup with health check endpoints, CORS middleware, multi-provider API Key Manager, and endpoint routing (/api/v1/meetings, /api/v1/chat).
LLM & Audio Pipeline	Completed	Structured schema parsing via Pydantic for summaries, action items, and key decisions. Multi-LLM provider abstraction.
Vector DB / RAG	Completed	Vector store integration for historical meeting search and retrieval-augmented conversational chat.
React Web App	Completed	Modular component structure with Tailwind dark-mode glassmorphism design, interactive modals (ConfirmModal, DecisionItem), task management views, and chat interface.
Chrome Extension	Completed	Manifest V3 build system configured, background service worker, popup UI, side panel integration, and dynamic icon generation scripts.
Configuration & Security	Completed	Environment-based configuration (.env), zero hardcoded secrets, strict input validation, and proper CORS restrictions.
Unit Testing & QA	Active / Ongoing	Unit tests implemented for backend modules (tests/) and frontend components (Popup.test.tsx, UI component tests). Full integration test pass ongoing.



8.Frontend:



 Action Items 6

☒ **Fix font sizing issue** MEDIUM Pending

Fix the font sizing issue on the mobile reports page as requested by the client. [Priority: High]

 Owner: Sarah

 Deadline: 2026-08-14

☒ **Color palette revision** MEDIUM Pending

Update the color palette to be more corporate and less playful, and run it through two rounds of client approval. [Priority: Medium]

 Owner: Sarah

 Deadline: 2026-08-18

☒ **Share finalized server requirements** MEDIUM Pending

Share the finalized server requirements from the client's server team. [Priority: High]

 Owner: Bilal

 Deadline: 2026-08-12

☒ **Backend API work** MEDIUM Pending

Complete the core API endpoints. [Priority: High]

